

E15 — Combination Sum (Backtracking / Recursion)

Experiment: 15 | LeetCode: #39 | CO: CO3, CO4 | BT Level: BT5

Problem Statement

Given an array of **distinct** integers `candidates` and a target integer `target`, return a list of all **unique combinations** of `candidates` where the chosen numbers sum to `target`. You may return the combinations in any order.

The **same** number may be chosen from `candidates` an **unlimited number of times**. Two combinations are unique if the frequency of at least one of the chosen numbers is different.

Example 1:

Input: `candidates = [2,3,6,7], target = 7`
Output: `[[2,2,3],[7]]`

Example 2:

Input: `candidates = [2,3,5], target = 8`
Output: `[[2,2,2,2],[2,3,3],[3,5]]`

Example 3:

Input: `candidates = [2], target = 1`
Output: `[]`

Constraints:

- $1 \leq \text{candidates.length} \leq 30$
 - $2 \leq \text{candidates}[i] \leq 40$
 - All elements of `candidates` are **distinct**
 - $1 \leq \text{target} \leq 40$
-

Concept: Backtracking

Backtracking is a systematic way to enumerate all possible solutions by building candidates incrementally and abandoning (backtracking) a candidate as soon as it is determined that it cannot lead to a valid solution.

Decision tree for `candidates = [2,3,6,7], target = 7`:

Start: `remaining = 7, path = []`

Choose 2: `remaining = 5, path = [2]`

Choose 2: `remaining = 3, path = [2,2]`

```

Choose 2: remaining = 1, path = [2,2,2]
  Choose 2: remaining = -1 → PRUNE (negative)
    Choose 3: remaining = -2 → PRUNE
      Choose 3: remaining = 0 → FOUND [2,2,3] ✓
        Choose 6: remaining = -3 → PRUNE
      Choose 3: remaining = 2, path = [2,3]
        Choose 3: remaining = -1 → PRUNE
          Choose 6: remaining = -1 → PRUNE
        Choose 3: remaining = 4, path = [3]
          Choose 3: remaining = 1, path = [3,3]
            Choose 3: remaining = -2 → PRUNE
              Choose 6: remaining = -2 → PRUNE
            Choose 6: remaining = 1, path = [6]
              Choose 6: remaining = -5 → PRUNE
            Choose 7: remaining = 0 → FOUND [7] ✓

```

Result: [[2,2,3],[7]]

Key pruning rule: Only consider candidates at index $\geq$ start to avoid duplicate combinations (e.g., [3,2] and [2,3] would be the same combination).

Solution 1: Backtracking (Standard)

```

def combinationSum(candidates, target):
    """
    Backtracking with pruning.
    Time: O(n^(t/m)) where t = target, m = min candidate value
    Space: O(t/m) for recursion depth
    """
    result = []

    def backtrack(start, remaining, path):
        if remaining == 0:
            result.append(list(path)) # Found a valid combination
            return
        if remaining < 0:
            return # Prune: exceeded target

        for i in range(start, len(candidates)):
            path.append(candidates[i])
            backtrack(i, remaining - candidates[i], path) # i, not i+
            path.pop() # Undo the choice (backtrack)

    backtrack(0, target, [])
    return result

```

Solution 2: Backtracking with Sorting (Early Termination)

Sorting the candidates allows breaking the loop early when a candidate exceeds the remaining target.

```

def combinationSum_optimized(candidates, target):
    """
    Sort first for early loop termination.
    Time:  $O(n \log n + n^{(t/m)})$  | Space:  $O(t/m)$ 
    """
    candidates.sort()    # Sort for pruning
    result = []

    def backtrack(start, remaining, path):
        if remaining == 0:
            result.append(list(path))
            return

        for i in range(start, len(candidates)):
            if candidates[i] > remaining:
                break    # All remaining candidates also exceed → stop
            path.append(candidates[i])
            backtrack(i, remaining - candidates[i], path)
            path.pop()

    backtrack(0, target, [])
    return result

```

Detailed Trace

candidates = [2, 3, 5], target = 8

Sorted candidates: [2, 3, 5]

```

backtrack(start=0, rem=8, path=[])
|
├─ Pick 2 → backtrack(0, 6, [2])
|   └─ Pick 2 → backtrack(0, 4, [2,2])
|       └─ Pick 2 → backtrack(0, 2, [2,2,2])
|           └─ Pick 2 → backtrack(0, 0, [2,2,2,2]) → FOUND ✓
|               └─ Pick 3 → rem=2-3<0 → BREAK
|                   └─ Pick 3 → backtrack(1, 1, [2,2,3])
|                       └─ Pick 3 → rem=1-3<0 → BREAK
|                           └─ Pick 5 → rem=4-5<0 → BREAK
|                               └─ Pick 3 → backtrack(1, 3, [2,3])
|                                   └─ Pick 3 → backtrack(1, 0, [2,3,3]) → FOUND ✓
|                                       └─ Pick 5 → rem=3-5<0 → BREAK
|                                           └─ Pick 5 → backtrack(2, 1, [2,5])
|                                               └─ Pick 5 → rem=1-5<0 → BREAK
|
├─ Pick 3 → backtrack(1, 5, [3])
|   └─ Pick 3 → backtrack(1, 2, [3,3])
|       └─ Pick 3 → rem=2-3<0 → BREAK
|           └─ Pick 5 → backtrack(2, 0, [3,5]) → FOUND ✓
|
└─ Pick 5 → backtrack(2, 3, [5])

```

└─ Pick 5 → rem=3-5<0 → BREAK

Result: [[2,2,2,2], [2,3,3], [3,5]] ✓

Variant: Combination Sum II (LC #40) — No Duplicates, Each Used Once

When candidates may contain duplicates and each element can only be used once:

```
def combinationSum2(candidates, target):
    """
    Each candidate used at most once; skip duplicates at the same level
    """
    candidates.sort()
    result = []

    def backtrack(start, remaining, path):
        if remaining == 0:
            result.append(list(path))
            return

        for i in range(start, len(candidates)):
            if candidates[i] > remaining:
                break
            # Skip duplicates at the same recursion level
            if i > start and candidates[i] == candidates[i - 1]:
                continue
            path.append(candidates[i])
            backtrack(i + 1, remaining - candidates[i], path)    # i+1:
            path.pop()

    backtrack(0, target, [])
    return result
```

Complexity Summary

Approach	Time	Space
Backtracking (unsorted)	$O(n^{(t/m)})$	$O(t/m)$
Backtracking (sorted)	$O(n^{(t/m)})$ with early exit	$O(t/m)$

Where:

- n = number of candidates
 - t = target value
 - m = minimum candidate value
-

Key Takeaways

- **Backtracking** = **DFS** + **Undo**: explore a choice, recurse, then undo the choice before trying the next.
- Pass `start` index (not `i+1`) when the same element can be reused — this avoids duplicates while allowing repetition.
- **Sorting** + **early break** significantly prunes the search tree in practice.
- The `path.pop()` (undo step) is critical: it restores the state before the loop tries the next candidate.
- Backtracking is the foundation for solving permutations, subsets, N-Queens, Sudoku, and many NP-complete problems.